

RAGNexus — Intelligent Document Q&A Assistant

-Vedant Shimpi

1. Selected SDG and Reason for Selection

Selected SDG: SDG 4 - Quality Education

What SDG 4 focuses on: SDG 4 aims to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all. It targets access to education, quality of learning, and the availability of resources for students across all age groups and economic backgrounds.

Why this issue is important: Education is the foundation of human development. Yet millions of students around the world, especially in developing countries, face serious barriers to quality learning. These barriers are not always physical. Even students who have access to schools and universities often struggle because:

- Academic content is dense, complex, and time-consuming to read
- Research papers and textbooks are difficult to navigate without expert guidance
- Tutors and academic support are expensive or unavailable
- Students waste hours searching for specific information buried inside long documents

In a world where knowledge is increasingly digital, the problem is not a lack of content, it is the lack of tools to make that content accessible and understandable to everyone.

Why I selected this SDG: I chose SDG 4 because technology has the power to democratize education. AI tools like RAGNexus can give every student regardless of their location, income, or access to tutors the ability to interact with any document intelligently and get instant, accurate answers. This directly supports the goal of quality, inclusive education for all.

2. Problem Statement

Students, researchers, and learners today have access to enormous amounts of digital content textbooks, research papers, lecture notes, government reports, and online articles. However, extracting specific, reliable information from these documents is

extremely time-consuming and difficult. Reading a 200-page research paper just to answer one question is not practical. There is no easy way for a student to simply ask a document a question and get a direct, accurate answer with proof of where the answer came from.

Where does this problem exist? This problem exists globally, but is especially severe in:

- Universities and colleges where students handle large volumes of reading
- Rural and semi-urban areas where access to qualified tutors and academic support is limited
- Developing countries where educational infrastructure is still growing
- Self-learners and working professionals who need quick access to specific knowledge

Who is affected?

- University and college students handling large volumes of academic content
- Researchers who need to cross-reference multiple papers and sources
- Teachers and educators preparing study material
- Professionals needing to query technical documents, manuals, or reports
- Anyone in a low-resource environment without access to personal tutors

Why is it serious? When students cannot efficiently extract knowledge from documents, they either spend disproportionate amounts of time reading, or they give up and rely on shallow summaries that miss critical details. This leads to poor academic performance, reduced research quality, and widening educational inequality between those who can afford expert guidance and those who cannot.

What could happen if it is not solved? Without tools to make documents accessible and queryable, the knowledge gap between privileged and underprivileged learners will continue to grow. Students without access to tutors or academic support will fall further behind. The potential of millions of capable students will remain unrealized simply because they lacked the right tools to learn effectively.

3. Proposed Solution

What is RAGNexus? RAGNexus is a full-stack, AI-powered intelligent document assistant built on a Retrieval-Augmented Generation (RAG) architecture. It allows any user to upload a document – a PDF, a text file, a CSV, or even a URL – and then have a natural conversation with that document. The user can ask any question in plain

English and receive a precise, grounded answer along with the exact source chunks the answer was drawn from.

How the system works:

1. The user uploads a document or pastes a URL into the interface
2. The system automatically reads, splits, and embeds the document into a vector database
3. When the user asks a question, the system searches the document semantically using both vector search and keyword search
4. The most relevant chunks are selected and passed to the Groq LLM
5. The LLM generates a grounded answer using only the retrieved content
6. The answer is returned to the user along with source citations so they can verify it

How it solves the problem: RAGNexus eliminates the need to manually read through long documents. A student can upload their entire textbook and ask specific questions chapter by chapter. A researcher can upload ten papers and query across all of them. A self-learner in a remote area with no tutor can get the same quality of guided learning that a student in a well-resourced institution gets. Every answer comes with sources, so users can trust the output and learn to read critically.

Who will use the application:

- Students at all levels, school, undergraduate, postgraduate
 - Researchers and academics
 - Teachers building question-answer materials from textbooks
 - Professionals querying technical manuals or policy documents
 - Self-learners and lifelong learners without access to formal academic support
-

4. Project Features

Feature 1 — Document Ingestion (URL, PDF, TXT, CSV) Users can upload any document type or paste a URL. The system automatically fetches, parses, and processes the content into searchable chunks. This means a student can load anything from a research paper PDF to a Wikipedia article URL and start asking questions immediately.

Feature 2 — Intelligent Question & Answer with Source Citations Users type any natural language question and receive a precise answer generated directly from the document content. Every answer includes numbered source citations showing exactly which part of the document was used, allowing users to verify and read further if needed.

Feature 3 — Hybrid Retrieval (Vector + Keyword Search) The system combines dense semantic vector search with BM25 keyword search. This means it finds relevant content even when the exact words in the question don't appear in the document — it understands meaning, not just terms.

Feature 4 — HyDE Query Expansion Before searching, the system generates a hypothetical answer to the question and uses that for retrieval. This significantly improves the relevance of retrieved chunks, especially for complex academic questions.

Feature 5 — Cross-Encoder Reranking After initial retrieval, a cross-encoder model re-reads each candidate chunk alongside the question and re-scores them for accuracy. This ensures only the most relevant content is passed to the LLM for answer generation.

Feature 6 — Persistent Chat Sessions with History Every conversation is saved to a database under a unique session ID. Users can return to a previous session, review their question history, and continue where they left off making it suitable for ongoing research or study sessions.

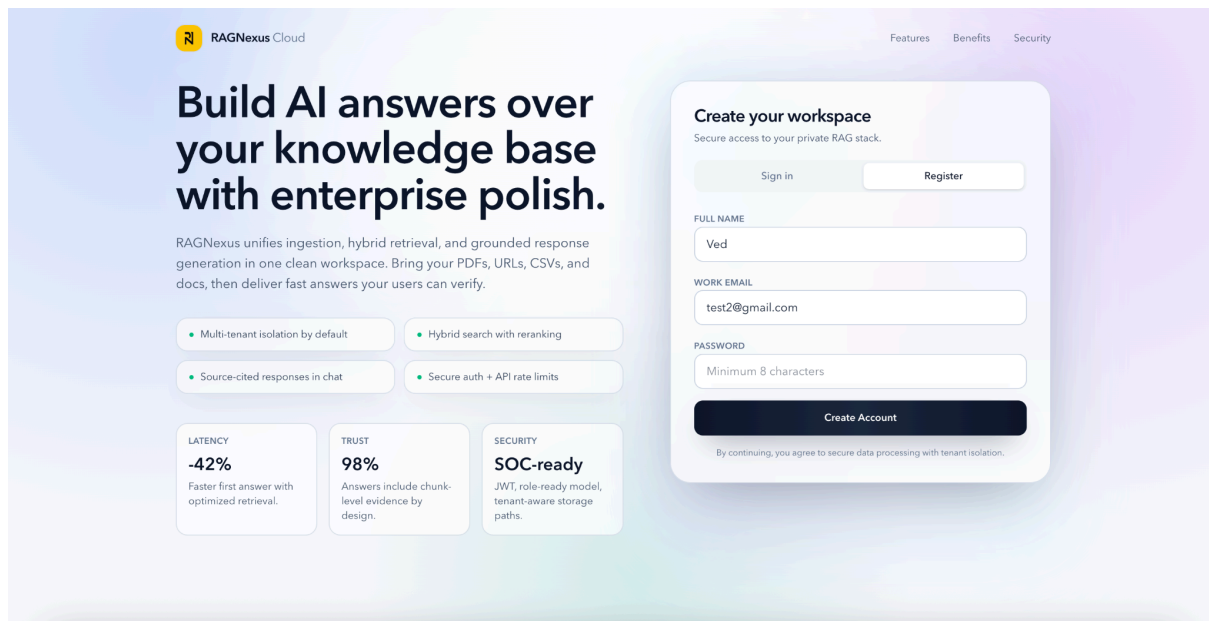
Feature 7 — Basic Chat Fallback Mode When no document is loaded, the chatbot switches to a general conversational mode powered by the Groq LLM. This means users can ask general knowledge questions, get explanations of concepts, or use it as a standard AI assistant even without uploading a document.

Feature 8 — React Frontend Chat Workspace A clean, responsive chat interface built with React and Vite. Users can manage documents, view source citations inline, switch between sessions, and interact with the chatbot in a familiar chat-style UI.

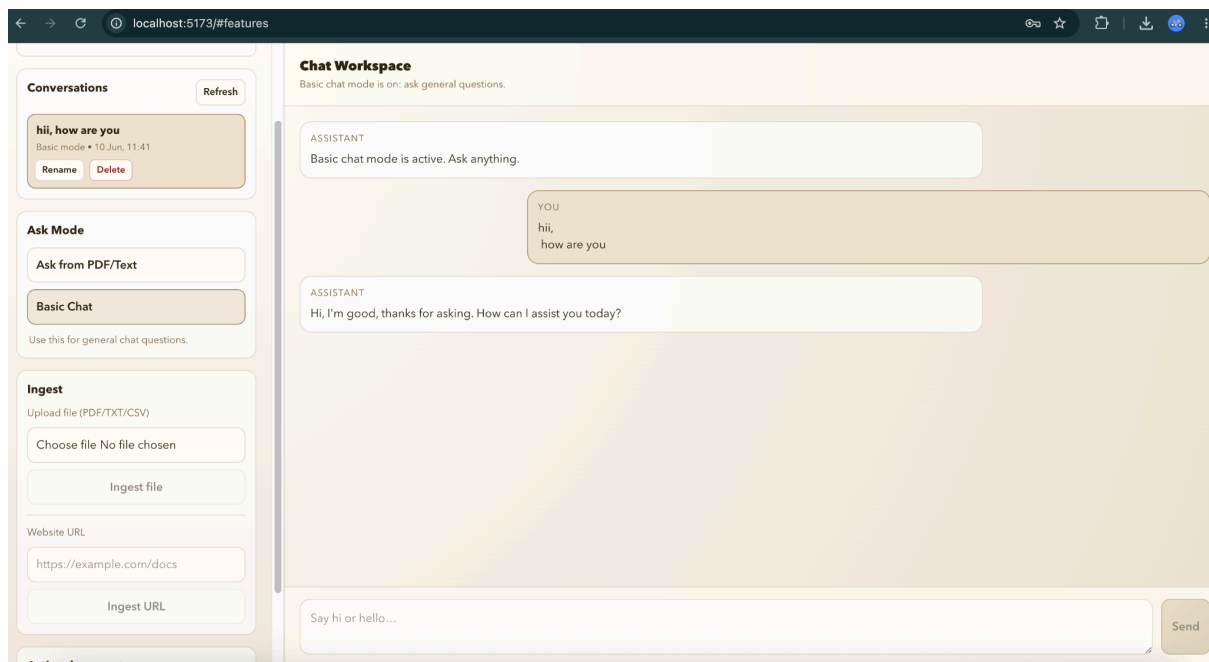
5. Technology Stack

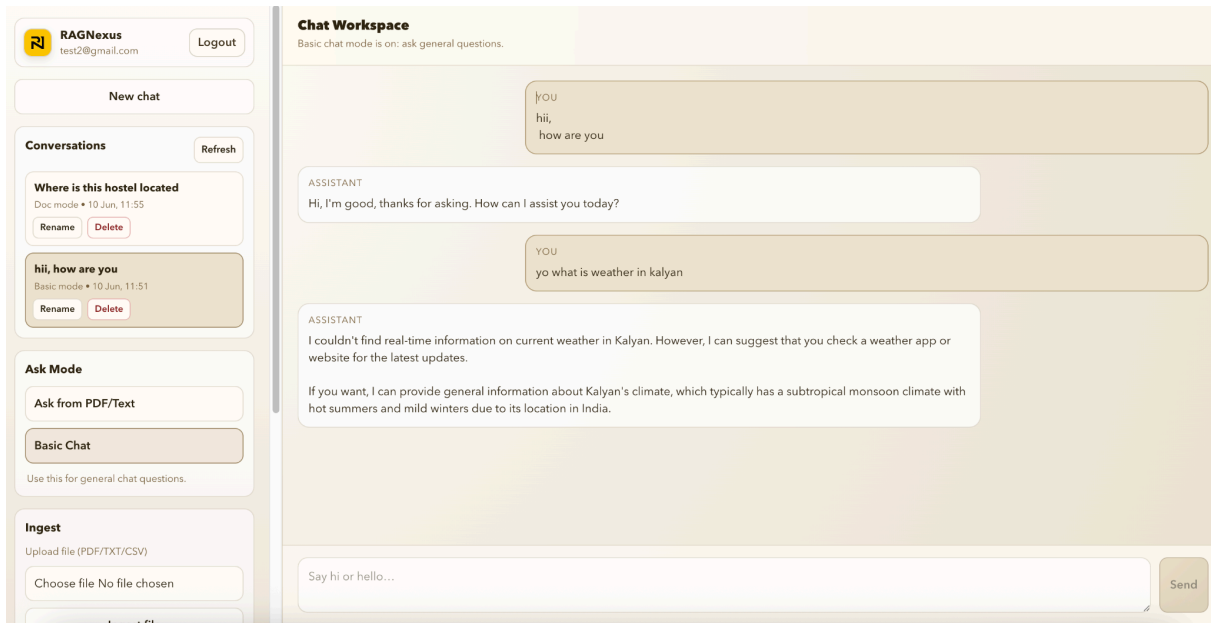
Category	Technology
Generative AI / LLM	Groq API — llama-3.1-8b-instant
RAG Framework	Custom pipeline (HyDE + Hybrid + Reranker)
Backend Framework	FastAPI (Python 3.10+)
Frontend	React + Vite (Node.js 18+)
Vector Database	Qdrant
Relational Database	PostgreSQL / Supabase
Embeddings Model	sentence-transformers (all-MiniLM-L6-v2)
Keyword Retrieval	BM25 (rank_bm25)
Reranker Model	Cross-Encoder (sentence-transformers)
ORM	SQLAlchemy
Evaluation (optional)	RAGAs
Containerization	Docker (for Qdrant)
Version Control	Git + GitHub

6. Project Screenshots

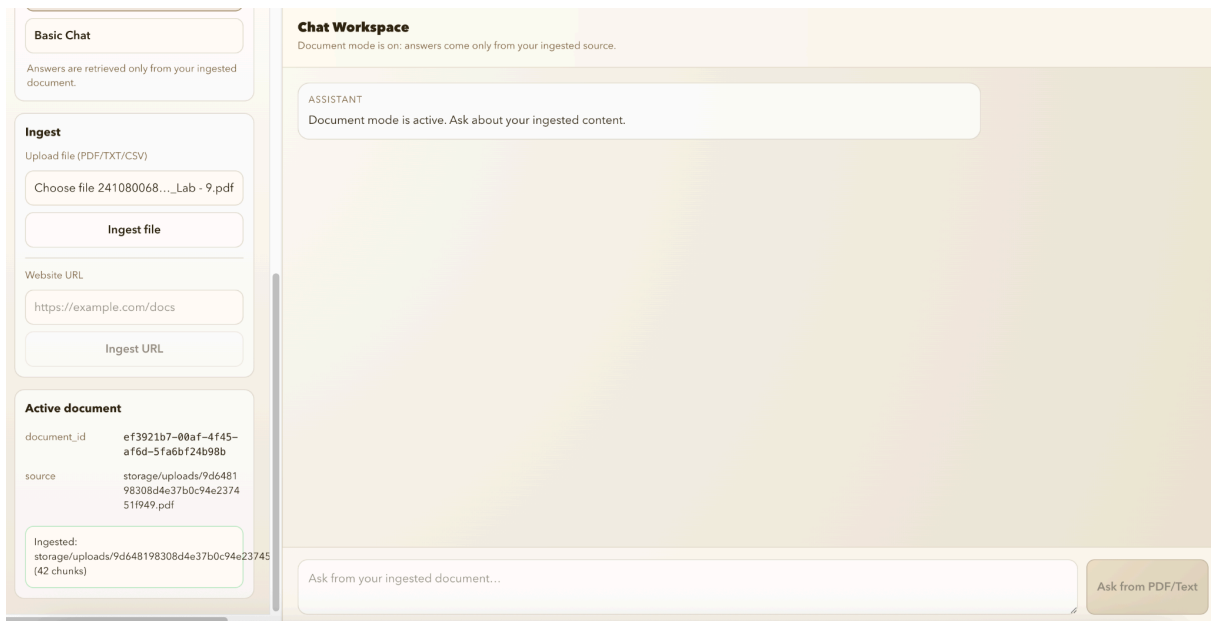


There are 2 options, one for basic answer retrieval and other for retrieval of answer from pdf or from website.





Ingesting the pdf



Asking Questions from pdf

Ask Mode

Ask from PDF/Text

Basic Chat

Answers are retrieved only from your ingested document.

Ingest

Upload file (PDF/TXT/CSV)

Choose file 241080068..._Lab - 9.pdf

Ingest file

Website URL

https://example.com/docs

Ingest URL

Active document

document_id

ef3921b7-00af-4f45-af6d-5fa6bf24b98b

source

storage/uploads/9d648198308d4e37b0c94e237451f949.pdf

ASSISTANT

Document mode is active. Ask about your ingested content.

YOU

What is subnetting

ASSISTANT

Subnetting is the technique of dividing a larger network into smaller sub-networks [1][2].

Sources

storage/uploads/9d648198308d4e37b0c94e237451f949.pdf

page 1

web hosting to other devices connected in the network.

Core Networking Concepts

● Subnetting:

Subnetting is the technique of dividing a larger network into smaller

storage/uploads/9d648198308d4e37b0c94e237451f949.pdf

page 1

sub-networks. This improves network efficiency, reduces unnecessary traffic, and enhances security.

● Default Gateway:

The default gateway is the IP address of the router interface that allows

storage/uploads/9d648198308d4e37b0c94e237451f949.pdf

page 2

● Borrowed bits: 2 (from /24 to /26)

● Number of subnets: 4

● Usable hosts per subnet: 62

This division helps in better utilization of IP addresses and minimizes broadcast traffic.

Now ingesting below URL

Website URL - <https://react.dev/learn/tutorial-tic-tac-toe>

Ask Mode

Ask from PDF/Text

Basic Chat

Answers are retrieved only from your ingested document.

Ingest

Upload file (PDF/TXT/CSV)

Choose file 241080068..._Lab - 9.pdf

Ingest file

Website URL

https://react.dev/learn/tutorial-tic-tac-

Ingest URL

Active document

document_id

d57fcc80-592b-47e4-90d4-f7696de606a7

source

https://react.dev/learn/tutorial-tic-tac-toe

YOU

How a make a tic-tac-toe game

ASSISTANT

To make a tic-tac-toe game, you need to lift state up from individual Square components to the Board component [2]. This involves storing a part of the game's state with each Square component, and the Board component would need to somehow know the state of each of the 9 Square components [2].

Here are the steps to make a tic-tac-toe game:

- Lift state up from individual Square components to the Board component
- Store a part of the game's state with each Square component
- Have the Board component check the state of each Square component
- Check if a player has won in the Board component's handleClick function (at the same time checking if a square already has an X or an O) [3]

The following benefits and features are present in the game:

None mentioned in the provided excerpts.

The game has the following features:

- Indicates when a player has won the game [1]
- Stores a game's history as a game progresses [1]
- Allows players to click on squares to play the game [3]

The benefits and features of the game include:

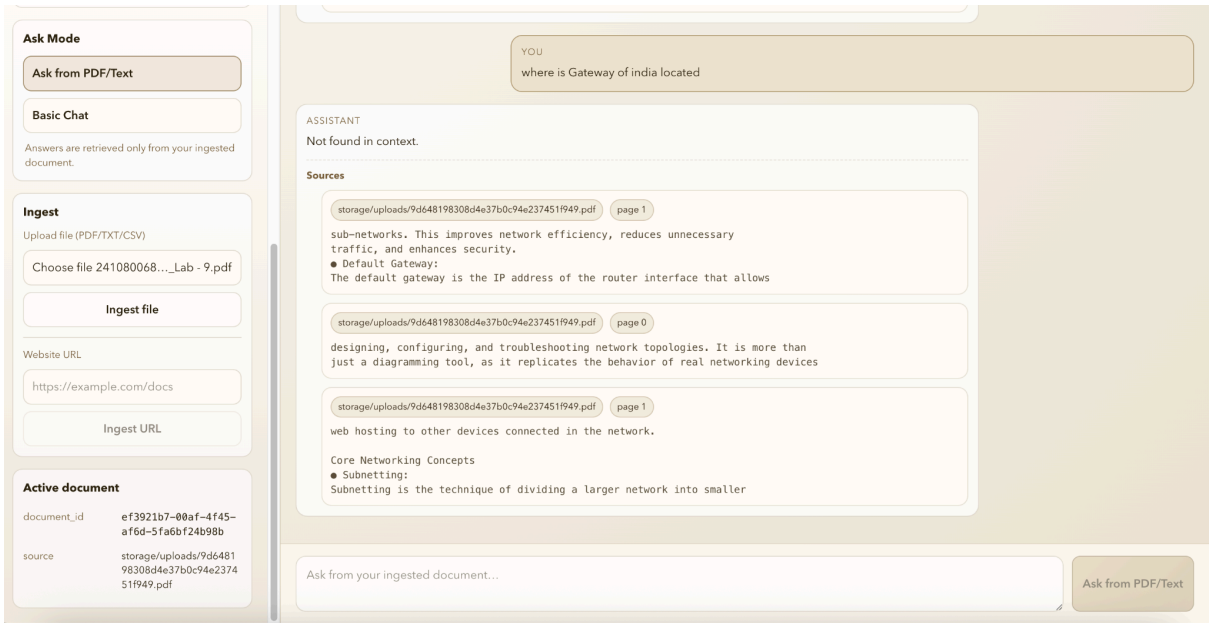
- Playing tic-tac-toe [1]
- Indicating a player's win [1]
- Storing a game's history [1]

Sources

https://react.dev/learn/tutorial-tic-tac-toe

Lets you play tic-tac-toe, Indicates when a player has won the game,

If particular content is not there in pdf, it doesn't answers that.



7. Open Source Repository

GitHub Repository Link -	https://github.com/VedWay/RAGNexus
Repository Name -	RAGNexus

8. Future Scope

- 1. Mobile Application** Develop a mobile version of RAGNexus for Android and iOS using React Native. This would allow students in remote areas to upload documents and query them directly from their smartphones without needing a laptop or desktop.
- 2. Multi-Language Support** Add support for regional and international languages so students who are not comfortable in English can interact with documents in Hindi, Marathi, Spanish, Arabic, and other languages. This would massively expand accessibility in developing countries.

- 3. Multi-Document Querying** Enable users to upload multiple documents simultaneously and ask questions that span across all of them. This would be especially useful for researchers comparing findings across several papers.
 - 4. Voice Input and Output** Integrate speech-to-text and text-to-speech so students can speak their questions and hear answers read aloud. This would benefit visually impaired users and students who prefer audio-based learning.
 - 5. Classroom & LMS Integration** Integrate RAGNexus with Learning Management Systems like Moodle, Google Classroom, or Canvas so teachers can upload course materials and students can query them directly within their existing study environment.
 - 6. Personalized Learning Paths** Use interaction history and quiz-style questioning to identify knowledge gaps and suggest specific sections of a document for the student to revisit, creating a personalized adaptive learning experience.
 - 7. Offline Mode** Develop a lightweight offline version using smaller on-device models so students in areas with poor internet connectivity can still use the tool with pre-loaded documents.
 - 8. Advanced Evaluation Dashboard** Build a teacher-facing dashboard using RAGAs metrics where educators can evaluate the quality of AI responses, track student queries, and identify commonly misunderstood topics.
-

Conclusion

RAGNexus directly addresses SDG 4 Quality Education by making knowledge accessible to anyone with a document and a question. By combining state-of-the-art retrieval techniques, a powerful language model, and a clean user interface, it serves as an AI tutor that any student anywhere in the world can use for free, without a human teacher, and without spending hours reading dense material. The project demonstrates real-world AI application development skills including RAG pipeline design, vector search, LLM integration, full-stack development, and conversational AI all applied toward a meaningful global challenge.